PROYECTO

SISTEMA DE GESTIÓN DE MANTENIMIENTO ELÉCTRICO

José Yahir Leonardo Ojer Barrios
Registro Académico: 202505578
Gustavo Angel Gómez Carreto
Registro Académico: 202505007
Mario David Hernández Calmo
Registro Académico: 202501990

Curso: 0769 - Introducción a la Programación de Computadoras
Universidad de San Carlos de Guatemala (USAC)

Resumen—En el ámbito de la ingeniería y la gestión industrial, la administración eficiente del mantenimiento de infraestructura y equipos eléctricos constituye un factor crítico para asegurar la continuidad operativa, mitigar riesgos de fallas inesperadas y optimizar los costos de reparación. Ante esta necesidad, se desarrolló el proyecto 'Sistema de Gestión de Mantenimiento Eléctrico', una solución informática diseñada para automatizar y centralizar el control de activos eléctricos, así como el registro detallado de las intervenciones preventivas y correctivas.

Para la concepción y estructuración de este software, se llevó a cabo un proceso sistemático de análisis y diseño computacional. Inicialmente, se elaboró el pseudocódigo y el algoritmo del programa, permitiendo definir la lógica de procesamiento, las secuencias de decisión, el flujo de datos y las reglas de validación necesarias. De manera complementaria, se construyó el diagrama de flujo correspondiente, proporcionando un modelo visual estructurado que representa la interacción entre las diferentes funcionalidades de la aplicación.

Este sistema servirá como una herramienta administrativa que permitirá a los operadores e ingenieros gestionar un inventario actualizado de equipos, programar tareas de mantenimiento preventivo para evitar el deterioro prematuro de los activos, registrar eventos de mantenimiento correctivo ante imprevistos técnicos y consultar historiales consolidados. De este modo, la aplicación contribuye a la sistematización del trabajo, garantizando la persistencia ordenada de los registros y facilitando la toma de decisiones informadas.

I. MARCO TEÓRICO

II.

III. II. OBJETIVOS DEL PROYECTO

A. Objetivo General

Desarrollar e implementar un software de escritorio funcional para la gestión del mantenimiento eléctrico, fundamentado en una planificación algorítmica previa (pseudocódigo y diagrama de flujo), que permita optimizar la administración de equipos y sistematizar el registro de mantenimientos preventivos y correctivos.

B. Objetivos Específicos

- 1. Planificación Algorítmica: Diseñar la lógica del programa mediante algoritmos, pseudocódigo y diagramas de flujo que representen gráficamente la secuencia lógica y el procesamiento de la información antes de la fase de codificación.
- 2. Control Centralizado de Inventario: Sistematizar el control de activos eléctricos facilitando el registro de equipos, sus ubicaciones y estados operativos dentro de un entorno centralizado y accesible.
- 3. Gestión de Intervenciones Técnicas: Administrar de forma estructurada las intervenciones de mantenimiento preventivo y correctivo, permitiendo catalogar tipos de servicios, frecuencias, costos asociados y diagnósticos técnicos.
- 4. Integridad y Consulta de Información: Proporcionar mecanismos de validación de datos para prevenir inconsistencias en la información ingresada y permitir la consulta organizada del historial de servicios realizados.

Enlace al repositorio de GitHub:

https://github.com/YOBI17/PROYECTO/tree/main/Proyecnto_Final

El proyecto consiste en una aplicación desarrollada en Python con la librería Tkinter para gestionar mantenimientos eléctricos (preventivos y correctivos). Aplica Programación Orientada a Objetos (POO) y almacena la información de manera persistente en archivos de texto plano (.txt) en una carpeta local datos/. El programa permite registrar equipos, asignar servicios y consultar historiales en tiempo real.

Objetivo General

Desarrollar e implementar un sistema de software modular en Python con interfaz gráfica y persistencia de datos local que permita organizar, registrar y consultar la información de clientes, equipos e intervenciones de mantenimiento eléctrico.

Objetivos Específicos

1. Aplicar los principios de la Programación Orientada a Objetos mediante el uso de clases, atributos y mecanismos de herencia para modelar las entidades de mantenimiento preventivo y correctivo.

2. Diseñar una interfaz gráfica intuitiva mediante Tkinter que contemple la validación de formularios, el manejo preventivo de excepciones y la limpieza de campos tras cada interacción.

3. Estructurar un módulo de almacenamiento en archivos de texto plano (.txt) que garantice la lectura y escritura persistente de los registros del sistema.

ALGORITMO

Inicio y preparación

1. **Inicio.**

2. **Ejecutar main.py,** que llama a la función iniciar_aplicacion() de interfaz.py.

3. **Crear** la ventana principal de Tkinter.

4. **Verificar la carpeta de datos** con asegurar_directorio(): si la carpeta datos/ no existe, crearla; si ocurre un error, mostrar un aviso.

5. **Cargar los equipos** con cargar_equipos():

   a) Si equipos.txt no existe, devolver una lista vacía y continuar.

   b) Si existe, leerlo línea por línea e ignorar las líneas vacías.

   c) Separar cada línea por el delimitador «|»; si no tiene 4 campos, avisar y omitirla.

   d) Crear un objeto Equipo con los campos y agregarlo a la lista.

6. **Cargar los mantenimientos preventivos** con cargar_preventivos(), con el mismo procedimiento del paso anterior, pero con 6 campos; convertir costo a decimal y frecuencia_dias a entero (si la conversión falla, avisar y omitir la línea).

7. **Cargar los mantenimientos correctivos** con cargar_correctivos(), con 6 campos y convirtiendo el costo a decimal.

8. **Construir** la pantalla de bienvenida y la ventana con pestañas: Equipos, Mantenimiento Preventivo, Mantenimiento Correctivo e Historial General.

9. **Llenar** las tablas y los combos con los datos cargados en los pasos 5, 6 y 7.

10. **Mostrar** la pantalla de bienvenida con el título del sistema y el botón «Ingresar al sistema».

11. **Esperar** el clic en «Ingresar al sistema»; al presionarlo, ocultar la bienvenida y mostrar las pestañas.

Ciclo principal

12. **Esperar una acción del usuario y decidir:**

    a) Si presiona «Registrar equipo», ir al paso 13.

    b) Si presiona «Registrar mantenimiento» (preventivo o correctivo), ir al paso 19.

    c) Si cambia de pestaña para consultar una tabla o el historial, mostrar la pestaña y volver al paso 12.

    d) Si cierra la ventana, ir al paso 32.

Registro de un equipo

13. **Leer** ID, nombre y ubicación de los cuadros de texto y el estado del combo, quitando los espacios sobrantes.

14. **Si algún campo está vacío,** mostrar el error «Campos vacíos» con messagebox.showerror y volver al paso 12.

15. **Si algún texto contiene «|»,** mostrar un error y volver al paso 12.

16. **Si ya existe un equipo con el mismo ID,** mostrar el error «ID duplicado» y volver al paso 12.

17. **Crear el objeto Equipo y guardarlo** con guardar_equipo(): asegurar la carpeta, reemplazar «|» y saltos de línea en los datos, unir los campos con «|» y escribir la línea al final de equipos.txt. Si ocurre un error de archivo (try-except), mostrar el error y volver al paso 12; si resulta bien, agregar el equipo a la lista en memoria.

18. **Limpiar** los cuadros de texto y el combo de estado, mostrar «Registro exitoso» e ir al paso 29.

Registro de un mantenimiento

19. **Leer** ID, equipo, servicio, fecha, costo y el dato propio del tipo (frecuencia en días si es preventivo, diagnóstico si es correctivo), quitando los espacios sobrantes.

20. **Si algún campo está vacío,** mostrar el error «Campos vacíos» y volver al paso 12.

21. **Si el ID o el diagnóstico contienen «|»,** mostrar un error y volver al paso 12.

22. **Si el ID ya existe** entre los mantenimientos preventivos y correctivos, mostrar el error «ID duplicado» y volver al paso 12.

23. **Si la fecha** no tiene el formato AAAA-MM-DD o no es una fecha real, mostrar el error y volver al paso 12.

24. **Si el costo** no es un número o es menor que 0, mostrar el error y volver al paso 12.

25. **Si es preventivo y la frecuencia** no es un entero mayor que 0, mostrar el error y volver al paso 12.

26. **Obtener** el id_equipo del equipo seleccionado y el código del servicio (por ejemplo P01 o C01), y crear el objeto MantenimientoPreventivo o MantenimientoCorrectivo.

27. **Guardar el mantenimiento** con guardar_preventivo() o guardar_correctivo(), que escriben una línea con los campos separados por «|» en preventivos.txt o correctivos.txt. Si ocurre un error de archivo, mostrar el error y volver al paso 12; si resulta bien, agregar el objeto a la lista en memoria.

28. **Limpiar** los cuadros de texto y los combos, mostrar «Registro exitoso» e ir al paso 29.

Actualización de tablas

29. **Actualizar los combos y las tablas:** reconstruir la lista de equipos de los combos y volver a

llenar las tablas de equipos, preventivos y correctivos con los datos en memoria.

30. **Actualizar el historial:** unir preventivos y correctivos en una sola lista, ordenarla por fecha, llenar la tabla del Historial General (tipo, ID, equipo, servicio, fecha, costo y detalle) y mostrar la cantidad de servicios y la suma de los costos.

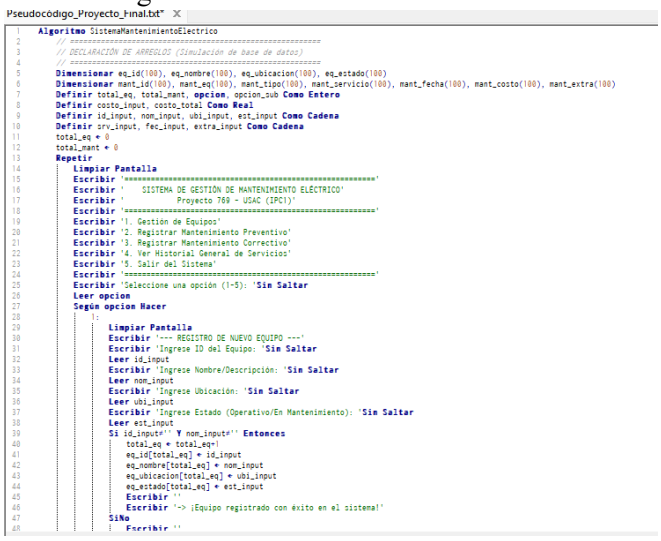
31. **Volver** al paso 12.

Cierre

32. **Cerrar** la ventana; los datos ya quedaron guardados en los archivos de la carpeta datos/.

33. **Fin.**

Pseudocodigo



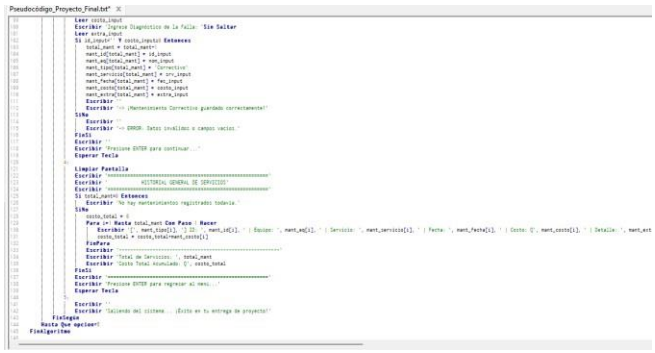
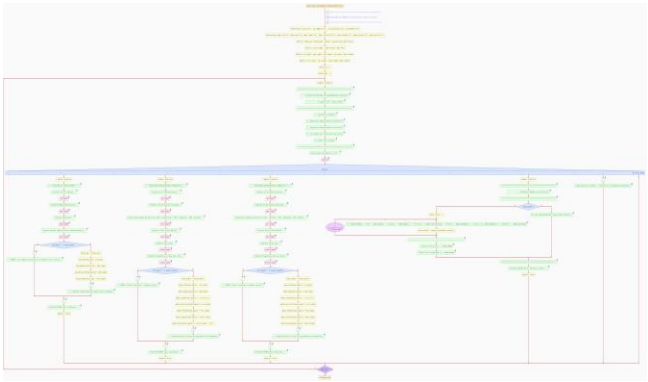
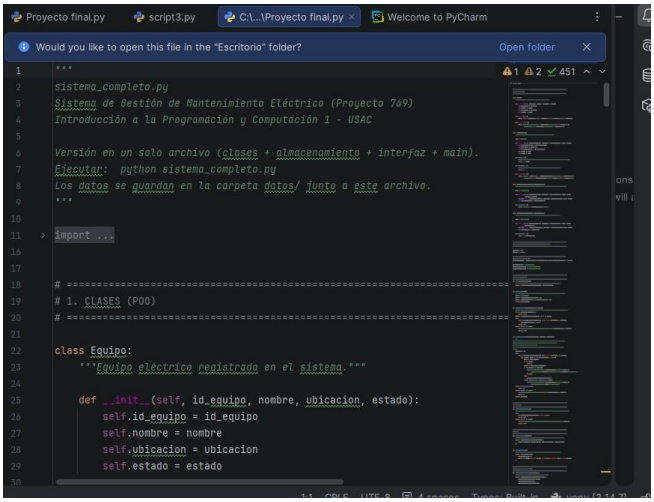
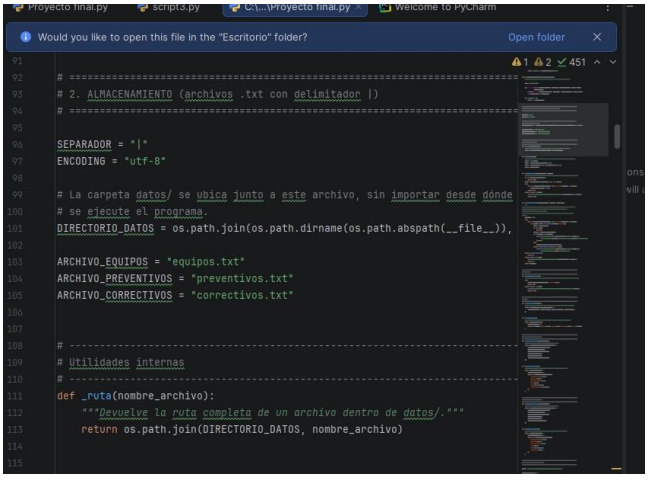


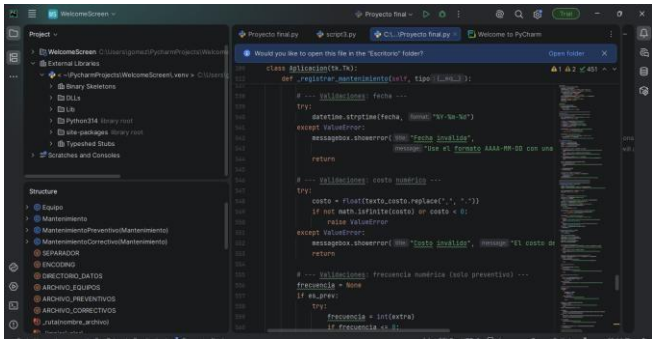
Diagrama de Flujo

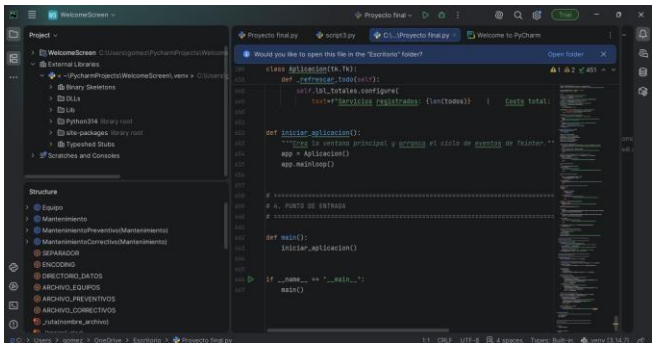


Codigo en Pycharm









Manual técnico

El sistema de de gestión de mantenimiento eléctrico es una aplicación para computadoras desarrollada en su totalidad en Python, este proyecto pertenece al curso de Introducción a la programación

Este programa permite registrar equipos eléctricos, administrar mantenimientos preventivos y correctivos. En el sistema nmos permite almacenar la información en archivos de texto y consultar un historial general de los servicios registrados. La aplicación utiliza programación orientada a objetos, para representar los equipos o/y mantenieientos mediante clases. Para la interfas grafica emplea tkinter, y para el almacenamiento de la información se uso mediante los archivos con la extensión de txt

Este sistema esta< desarrollado en un único archivo, este integra las clases y sus funciones, la interfaz grafica y los puntos de entrada del programa.

Estructura del programa

El código depende de cuatro secciones principales clases, almacenamiento, interfaz grafica y punto de entrada

Cada una cumple la función específica, están organizadas para permitir separar y ordenar la información y su almacenamiento, así mismo la presentación al usuario

- Clases= en este apartado se define los objetos que representan los equipos eléctricos y los diferentes tipos de mantenimiento.

- Almacenamiento= contiene las funciones encargadas de guardar y recuperar información mediante archivos de texto.

- Interfaz gráfica= construye las ventanas, formularios, tablas y botones con los que interactúa el usuario.

- Punto de entrada= inicia la aplicación y activa el ciclo de eventos de tkinter.

Tipos de mantenimiento

1- Clase de equipo
Este representa un equipo eléctrico registrado en el sistema.

2- Clase base Mantenimiento
La clase Mantenimiento representa la información común a todos los registros de mantenimiento.

3- Clase MantenimientoPreventivo
Esta clase hereda los atributos y métodos de Mantenimiento. Su finalidad es representar los servicios programados que se realizan periódicamente para prevenir fallas.

4- Clase MantenimientoCorrectivo

5- Esta clase también hereda de Mantenimiento, pero representa los servicios destinados a reparar fallas o problemas identificados en los equipos eléctricos. Su atributo adicional es el diagnostico; descripción del problema detectado o de la reparación correspondiente.
El atributo TIPO contiene el valor "Correctivo", y el método detalle() devuelve el diagnóstico registrado.

- ALMACENAMIENTO DE LA INFORMACION
El programa utiliza archivos de texto para conservar los registros entre ejecuciones. Por lo

tanto, los datos no se almacenan únicamente en la memoria mientras la aplicación está abierta.
La carpeta datos/ se crea junto al archivo del programa. Para determinar su ubicación se emplean las funciones os.path.abspath(), os.path.dirname() y os.path.join().
Esta forma de construir las rutas permite que los archivos de datos se localicen junto al programa, independientemente de la carpeta desde la cual se ejecute.

- ARCHIVOS UTILIZADOS

El sistema utiliza tres archivos principales:

equipos.txt: almacena los equipos eléctricos.

preventivos.txt: almacena los mantenimientos preventivos.

correctivos.txt: almacena los mantenimientos correctivos.

Cada registro se escribe en una línea. Los campos se separan mediante el carácter |, definido en la constante SEPARADOR.

- Funciones específicas de lectura y escritura

Las funciones generales se complementan con seis funciones específicas:

guardar_equipo() y cargar_equipos().

guardar_preventivo() y cargar_preventivos().

guardar_correctivo() y cargar_correctivos().

Las funciones de guardado reciben un objeto, extraen sus atributos y los escriben en el archivo correspondiente. Las funciones de carga recuperan los campos almacenados y reconstruyen los objetos. También convierten algunos datos al tipo apropiado: por ejemplo, el costo se convierte a float y la frecuencia preventiva a int.

Esta estructura evita duplicar la lógica de lectura y escritura para cada tipo de registro.

- Catálogos y constantes

El programa define constantes que centralizan valores utilizados en distintas partes del código.

Entre ellas se encuentran:

1- TITULO: nombre de la aplicación.
2- CATALOGO_PREVENTIVO: códigos y descripciones de los servicios preventivos.
3- CATALOGO_CORRECTIVO: códigos y descripciones de los servicios correctivos.
4- ESTADOS: estados disponibles para los equipos.
5- COLOR_PRINCIPAL y COLOR_ACENTO: colores empleados en la interfaz.
6- SEPARADOR y ENCODING: configuración del formato de los archivos.

Los catálogos permiten seleccionar servicios mediante códigos, como P01 o C01, y mostrar sus descripciones correspondientes.Esta organización evita repetir las descripciones de los servicios en diferentes secciones del programa y facilita su modificación.

- Interfaz gráfica con Tkinter

La interfaz gráfica permite utilizar el sistema mediante ventanas, formularios, botones, pestañas y tablas, sin necesidad de introducir comandos directamente en la consola.El programa utiliza tkinter para los componentes básicos y tkinter.ttk para componentes con estilos mejorados. También emplea messagebox para mostrar mensajes de error, advertencias informativas y confirmaciones.

- Clase Aplicacion

La clase Aplicacion hereda de tk.Tk, por lo que representa la ventana principal del sistema.

Su constructor, __init__(), realiza las siguientes operaciones:

Inicializa la ventana principal.

Configura el título y las dimensiones.

Establece los estilos visuales.

Comprueba la existencia de la carpeta de datos.

Carga los equipos y mantenimientos almacenados.

Inicializa las estructuras internas utilizadas por los formularios.

Construye la pantalla de bienvenida y la ventana principal.

Muestra inicialmente la pantalla de bienvenida.

Los registros cargados se mantienen en las listas self.equipos, self.preventivos y self.correctivos. Estas listas permiten consultar y actualizar los datos durante la ejecución.

- Pantalla de bienvenida

El método _crear_bienvenida() construye la pantalla inicial. Muestra el título del sistema, información del proyecto y un botón para ingresar. Al pulsar el botón, se ejecuta el método _mostrar() para ocultar la pantalla de bienvenida y mostrar la ventana principal.

El método _mostrar() administra la navegación entre los dos marcos principales de la aplicación.

- Ventana principal y pestañas

El método _crear_principal() construye la ventana de trabajo y organiza las funciones del sistema mediante cuatro pestañas:

1- Equipos: permite registrar equipos y consultar los registrados.
2- Mantenimiento Preventivo: permite registrar y visualizar mantenimientos preventivos.
3- Mantenimiento Correctivo: permite registrar y visualizar mantenimientos correctivos.
4- Historial General: reúne los servicios registrados y presenta el costo total acumulado.

- Funcionamiento del registro de equipos

El registro de equipos se realiza mediante el método _registrar_equipo().

El proceso comienza cuando el usuario completa los campos del formulario y pulsa el botón correspondiente. El programa obtiene los valores con el método .get() de cada campo y elimina los espacios externos mediante .strip().

Después, ejecuta las siguientes validaciones:

Comprueba que todos los campos estén completos.

Verifica que los campos de texto no contengan el carácter |, que se utiliza como separador en los archivos. Comprueba que el identificador del equipo no esté registrado previamente. Si alguna validación falla, se muestra un mensaje de error y se interrumpe el registro. Si los datos son válidos, se crea un objeto Equipo y se intenta guardar mediante guardar_equipo(). Cuando el guardado se realiza correctamente, el objeto se agrega a self.equipos, se actualizan las tablas y las listas desplegables, se limpian los campos del formulario y se muestra un mensaje de confirmación. Si no es posible guardar el registro, el programa muestra un error y no lo agrega a la lista de equipos en memoria.

- Funcionamiento del registro de mantenimientos

El método _crear_tab_mantenimiento() construye los formularios para los mantenimientos preventivos y correctivos.Ambos formularios comparten los campos de identificador, equipo, servicio, fecha y costo. La diferencia está en el último campo:

- Para el mantenimiento preventivo se solicita la frecuencia en días.
- Para el mantenimiento correctivo se solicita un diagnóstico.

El método _registrar_mantenimiento(tipo) procesa los datos de ambos formularios. El parámetro tipo determina qué clase debe utilizarse y en qué archivo se almacenará el registro.

- Creación y guardado del mantenimiento

Una vez superadas las validaciones, el programa crea un objeto de la clase correspondiente. Si el tipo es preventivo, construye un MantenimientoPreventivo y utiliza guardar_preventivo(). Si el tipo es correctivo, construye un MantenimientoCorrectivo y utiliza guardar_correctivo(). Cuando el guardado tiene éxito, el objeto se agrega a la lista correspondiente, se actualizan las tablas, se limpian los campos y se muestra un mensaje de confirmación. Si ocurre

un error al guardar, el programa informa al usuario y detiene el proceso.

- Flujo general de funcionamiento

1- El usuario ejecuta el archivo
2- Python ejecuta main() e inicia la aplicación.
3- Se crea la ventana principal y se cargan los registros existentes desde los archivos de texto.
4- Se muestra la pantalla de bienvenida.
5- El usuario ingresa al sistema y selecciona la pestaña que necesita.
6- Completa el formulario correspondiente y pulsa el botón de registro.
7- El programa valida los datos ingresados.
8- Si los datos son correctos, crea el objeto correspondiente y lo guarda en el archivo adecuado.
9- El registro se agrega a la lista en memoria y las tablas se actualizan.
10- El usuario puede consultar los equipos, los mantenimientos y el historial general.
11- Este flujo permite que la información registrada permanezca disponible después de cerrar y volver a abrir la aplicación.



